Streaming Production Patterns

Document Information

Field	Value
Document ID	SA-DP-014
Version	2.0
Last Updated	February 2026
Owner	Data Engineering
Status	Approved

Version History

Version	Date	Changes
2.0	Feb 2026	Prioritized Lakeflow SDP as primary streaming approach, added expectations patterns
1.0	Jan 2026	Initial streaming patterns

Golden Rules Summary

Rule ID	Rule	Severity	Description	WAF Pillar
ST-01	Lakeflow SDP First	Critical	Use Lakeflow SDP (formerly DLT) for all streaming workloads unless exception approved	Reliability
ST-02	Managed Checkpointing	Required	Use SDP's managed checkpointing for disaster recovery	Reliability
ST-03	Fixed Worker Count	Required	Disable auto-scaling for streaming workloads	Reliability
ST-04	Idempotent Operations	Required	Ensure all streaming operations are idempotent	Reliability

> **Cross-references:** Data quality expectations → see DQ-01. Jobs compute only → see CP-05.

Platform & Enterprise Prerequisites

> This document defines **solution patterns**. The following EA/PA rules are prerequisites.

Prerequisite	Rule IDs	Document
Unity Catalog managed tables	DL-01..05	12-unity-catalog-tables
DLT expectations for quality	DQ-01	09-data-quality
Jobs compute / cluster policies	CP-05	13-cluster-policies
Reliability patterns	REL-03	17-reliability-disaster-recovery
Serverless compute	SC-01..04	11-serverless-compute

Lakeflow Spark Declarative Pipelines (SDP)

> **Reference:** Lakeflow Spark Declarative Pipelines

Why Lakeflow SDP is the Recommended Approach

Rule ST-01: Use Lakeflow SDP for all streaming workloads unless exception approved

Lakeflow Spark Declarative Pipelines (SDP), formerly known as Delta Live Tables (DLT), is the **recommended approach for all streaming workloads** on Databricks.

```
┌──────────────────────────────────────────────────────────────────────────────────────────────┐
│                                STREAMING APPROACH DECISION TREE                                │
├──────────────────────────────────────────────────────────────────────────────────────────────┤
│                                                                                              │
│   START: New Streaming Workload                                                              │
│                                  │                                                           │
│                                  ▼                                                           │
│                        ┌──────────────────────┐                                              │
│                        │ Can use declarative  │                                              │
│                        │ SQL/Python?          │                                              │
│                        └──────────┬───────────┘                                              │
│                                   │                                                          │
│                          ┌────────┴────────┐                                                 │
│                          │                 │                                                 │
│                         YES               NO                                                 │
```

```
│                                                                                │
│            │              │
│            ▼              ▼
│
│    ┌─────────────────┐  ┌─────────────────┐
│    │ ✅ USE          │  │ Complex custom  │
│    │ LAKEFLOW SDP    │  │ logic required? │
│    │                 │  └────────┬────────┘
│    │ • Managed       │           │
│    │ • Checkpointed  │      ┌────┴────┐
│    │ • Open Source   │    YES        NO
│    │ • Expectations  │     │          │
│    └─────────────────┘     ▼          ▼
│                      ┌──────────┐  ┌────────────┐
│                      │ Structured│  │ Return to  │
│                      │ Streaming │  │ Lakeflow   │
│                      │ (Exception│  │ SDP        │
│                      │ Required) │  └────────────┘
│                      └──────────┘
│
│
│
└──────────────────────────────────────────────────────────────────────────────
────────────┘
```

Key Benefits of Lakeflow SDP

Benefit	Description	Business Value
Open Source	Built on Apache Spark Declarative Pipelines, avoiding vendor lock-in	Portability, community support
Managed Service	Databricks handles infrastructure, scaling, and recovery	Reduced operational burden
Automatic Checkpointing	Built-in state management for disaster recovery	RPO/RTO compliance

Benefit	Description	Business Value
Data Quality Expectations	Declarative quality rules with automatic enforcement	Data reliability
Unified Batch & Streaming	Same code handles both processing modes	Development efficiency
Automatic Schema Evolution	Handles schema changes without manual intervention	Operational resilience
Built-in Monitoring	Pipeline observability dashboard and metrics	Easier troubleshooting

Lakeflow SDP vs Raw Structured Streaming

Feature	Lakeflow SDP	Raw Structured Streaming
Checkpoint Management	✅ Automatic, managed	❌ Manual configuration
Data Quality	✅ Built-in Expectations	❌ Custom implementation
Schema Evolution	✅ Automatic	❌ Manual handling
Recovery from Failures	✅ Automatic restart	⚠ Requires job configuration
Monitoring	✅ Built-in dashboard	❌ Custom metrics setup
Cost Optimization	✅ Serverless option	❌ Manual cluster management
Code Complexity	✅ Declarative, simple	⚠ Imperative, complex
Production Readiness	✅ Built-in best practices	⚠ Manual implementation

Data Quality with Expectations

> **Reference:** Manage data quality with expectations

> **Note:** Data quality expectations are defined in DQ-01. The patterns below show streaming-specific implementation.

Lakeflow SDP provides built-in data quality enforcement through **Expectations** - declarative rules that validate data as it flows through the pipeline.

Expectation Types

Type	Behavior	Use Case
`@dlt.expect`	Log warning, continue processing	Non-critical rules, monitoring
`@dlt.expect_or_drop`	Drop invalid rows, continue	Data cleaning, filtering
`@dlt.expect_or_fail`	Fail pipeline on violation	Critical business rules
`@dlt.expect_all`	Apply multiple expectations	Complex validation

Type	Behavior	Use Case
`@dlt.expect_all_or_drop`	Drop if any expectation fails	Multi-rule filtering
`@dlt.expect_all_or_fail`	Fail if any expectation fails	Multi-rule critical validation

Expectation Patterns

Basic Expectations (Python):

```python
import dlt
from pyspark.sql.functions import col, current_timestamp

@dlt.table(
    name="silver_events",
    comment="Cleaned event data with quality expectations",
    table_properties={
        "quality": "silver",
        "delta.enableChangeDataFeed": "true"
    }
)
@dlt.expect("valid_event_id", "event_id IS NOT NULL")
@dlt.expect("valid_timestamp", "event_timestamp > '2020-01-01'")
@dlt.expect_or_drop("valid_user_id", "user_id IS NOT NULL AND user_id >
0")
@dlt.expect_or_fail("valid_amount", "amount >= 0")
def silver_events():
    return (
        dlt.read_stream("bronze_events")
        .withColumn("processed_at", current_timestamp())
    )
```

Multiple Expectations (Python):

```python
@dlt.table(name="silver_transactions")
@dlt.expect_all_or_drop({
    "valid_transaction_id": "transaction_id IS NOT NULL",
    "valid_customer_id": "customer_id IS NOT NULL",
    "valid_amount": "amount > 0",
    "valid_currency": "currency IN ('USD', 'EUR', 'GBP')"
})
def silver_transactions():
    return dlt.read_stream("bronze_transactions")
```

Expectations (SQL):

```sql
CREATE OR REFRESH STREAMING TABLE silver_events (
    CONSTRAINT valid_event_id EXPECT (event_id IS NOT NULL),
```

```
    CONSTRAINT valid_timestamp EXPECT (event_timestamp > '2020-01-01'),
    CONSTRAINT valid_user_id EXPECT (user_id IS NOT NULL AND user_id > 0)
ON VIOLATION DROP ROW,
    CONSTRAINT valid_amount EXPECT (amount >= 0) ON VIOLATION FAIL UPDATE
)
AS SELECT
    event_id,
    user_id,
    event_timestamp,
    amount,
    current_timestamp() AS processed_at
FROM STREAM(bronze_events);
```

Quarantine Pattern for Invalid Data

```
# Main table with strict expectations
@dlt.table(name="silver_orders")
@dlt.expect_all_or_drop({
    "valid_order_id": "order_id IS NOT NULL",
    "valid_customer": "customer_id IS NOT NULL",
    "valid_total": "total_amount > 0"
})
def silver_orders():
    return dlt.read_stream("bronze_orders")

# Quarantine table for invalid records
@dlt.table(
    name="quarantine_orders",
    comment="Invalid orders that failed quality checks"
)
def quarantine_orders():
    return (
        dlt.read_stream("bronze_orders")
        .filter(
            (col("order_id").isNull()) |
            (col("customer_id").isNull()) |
            (col("total_amount") <= 0)
        )
        .withColumn("quarantine_reason",
            when(col("order_id").isNull(), "NULL_ORDER_ID")
            .when(col("customer_id").isNull(), "NULL_CUSTOMER_ID")
            .when(col("total_amount") <= 0, "INVALID_AMOUNT")
        )
        .withColumn("quarantined_at", current_timestamp())
    )
```

Managed Checkpointing for Disaster Recovery

Rule ST-02: Use SDP's managed checkpointing for disaster recovery

Lakeflow SDP automatically manages checkpoints for streaming tables, providing:

Feature	Description
Automatic State Management	Checkpoints stored in managed location
Failure Recovery	Automatic restart from last checkpoint
Exactly-Once Semantics	Guaranteed message processing
Cross-Region DR	Checkpoints replicated (with DR configuration)

```
┌──────────────────────────────────────────────────────────────────────────────────────────────┐
│                                LAKEFLOW SDP CHECKPOINTING ARCHITECTURE                                │
│                                                                                                      │
│                                                                                                      │
│                                                                                                      │
│    ┌──────────────────┐        ┌──────────────────┐        ┌──────────────────┐                    │
│    │   SOURCE         │        │   STREAMING      │        │    TARGET        │                    │
│    │   (Kafka,        │  ──►   │    TABLE         │  ──►   │    TABLE         │                    │
│    │    S3, etc.)     │        │   (SDP)          │        │   (Delta)        │                    │
│    └──────────────────┘        └──────────┬───────┘        └──────────────────┘                    │
│                                           │                                                          │
│                                           ▼                                                          │
│                                                                                                      │
│                          ┌──────────────────────────────┐                                            │
│                          │   MANAGED CHECKPOINT         │                                            │
│                          │   ────────────────────       │                                            │
│                          │   • Offset tracking          │                                            │
│                          │   • State management         │                                            │
│                          │   • Failure recovery         │                                            │
│                          │   • Auto-replication (DR)    │                                            │
│                          └──────────────────────────────┘                                            │
│                                                                                                      │
│                                                                                                      │
│    On Failure:
```

```
│
│    ┌──────────────────────────────────────────────────────────────────┐
│
│    │ 1. Pipeline detects failure                                      │
│
│    │ 2. Reads last checkpoint                                         │
│
│    │ 3. Restarts from exact position                                  │
│
│    │ 4. No data loss, no duplicates                                   │
│
│    └──────────────────────────────────────────────────────────────────┘
│
│
│
└──────────────────────────────────────────────────────────────────────────────────────────────┘
```

Lakeflow SDP Asset Bundle Configuration

Serverless Pipeline (Recommended)

```yaml
resources:
  pipelines:
    streaming_pipeline:
      name: "[${bundle.target}] Event Streaming Pipeline"

      # Lakeflow SDP configuration
      catalog: ${var.catalog}
      schema: ${var.silver_schema}

      # Pipeline source code
      root_path: ../src/streaming_pipeline
      libraries:
        - notebook:
            path: ../src/streaming/silver_events.py
        - notebook:
            path: ../src/streaming/silver_transactions.py

      # Serverless compute (recommended)
      serverless: true

      # Photon for performance
      photon: true

      # Latest features
      channel: CURRENT

      # Streaming mode
      continuous: true  # For real-time streaming
```

```
      # OR
      # continuous: false  # For triggered/scheduled streaming

      # Advanced edition for expectations
      edition: ADVANCED

      # Pipeline configuration
      configuration:
        catalog: ${var.catalog}
        bronze_schema: ${var.bronze_schema}
        silver_schema: ${var.silver_schema}
        pipelines.enableTrackHistory: "true"

      # Notifications
      notifications:
        - alerts:
            - on-update-failure
            - on-update-fatal-failure
            - on-flow-failure
          email_recipients:
            - data-engineering@company.com

      # Tags for cost allocation
      tags:
        environment: ${bundle.target}
        layer: silver
        pipeline_type: streaming
        team: data-engineering
```

Classic Compute Pipeline (When Serverless Not Available)

```
resources:
  pipelines:
    streaming_pipeline_classic:
      name: "[${bundle.target}] Event Streaming Pipeline"

      catalog: ${var.catalog}
      schema: ${var.silver_schema}

      root_path: ../src/streaming_pipeline
      libraries:
        - notebook:
            path: ../src/streaming/silver_events.py

      # Classic compute configuration
      serverless: false
      photon: true

      clusters:
        - label: default
          num_workers: 4  # Fixed size - no auto-scaling!
```

```
          spark_conf:
            spark.databricks.cluster.profile: serverless
          custom_tags:
            team: data-engineering
            cost_center: CC-1234

      continuous: true
      edition: ADVANCED
```

Complete Lakeflow SDP Pipeline Example

Bronze to Silver Streaming Pipeline

```
# Databricks notebook source
# MAGIC %md
# MAGIC # Silver Layer Streaming Pipeline
# MAGIC Uses Lakeflow SDP (formerly DLT) for production-grade streaming

import dlt
from pyspark.sql.functions import col, current_timestamp, when, lit

# Get configuration
catalog = spark.conf.get("catalog")
bronze_schema = spark.conf.get("bronze_schema")
silver_schema = spark.conf.get("silver_schema")

#
==========================================================================
==
# STREAMING TABLE: Events
#
==========================================================================
==

@dlt.table(
    name="silver_events",
    comment="LLM: Cleaned event stream with quality validation. Source:
bronze_events",
    table_properties={
        "quality": "silver",
        "delta.enableChangeDataFeed": "true",
        "delta.enableRowTracking": "true",
        "delta.autoOptimize.optimizeWrite": "true",
        "delta.autoOptimize.autoCompact": "true",
        "layer": "silver",
        "domain": "events"
    },
    cluster_by_auto=True
)
@dlt.expect("event_id_not_null", "event_id IS NOT NULL")
```

```
@dlt.expect("valid_event_type", "event_type IN ('click', 'view',
'purchase', 'signup')")
@dlt.expect_or_drop("valid_user_id", "user_id IS NOT NULL")
@dlt.expect_or_fail("valid_timestamp", "event_timestamp <=
current_timestamp()")
def silver_events():
    """Stream events from bronze with quality checks."""
    return (
        dlt.read_stream("bronze_events")
        .select(
            col("event_id"),
            col("user_id"),
            col("event_type"),
            col("event_timestamp"),
            col("properties"),
            current_timestamp().alias("processed_at")
        )
    )

#
==============================================================================
==
# QUARANTINE TABLE: Invalid Events
#
==============================================================================
==

@dlt.table(
    name="quarantine_events",
    comment="Events that failed quality validation",
    table_properties={
        "quality": "quarantine",
        "layer": "silver"
    }
)
def quarantine_events():
    """Capture events that fail quality checks for remediation."""
    return (
        dlt.read_stream("bronze_events")
        .filter(
            (col("event_id").isNull()) |
            (~col("event_type").isin("click", "view", "purchase",
"signup")) |
            (col("user_id").isNull())
        )
        .withColumn("quarantine_reason",
            when(col("event_id").isNull(), "NULL_EVENT_ID")
            .when(~col("event_type").isin("click", "view", "purchase",
"signup"), "INVALID_EVENT_TYPE")
            .when(col("user_id").isNull(), "NULL_USER_ID")
            .otherwise("UNKNOWN")
        )
        .withColumn("quarantined_at", current_timestamp())
    )
```

```
#
==========================================================================
==
# MATERIALIZED VIEW: Event Aggregates
#
==========================================================================
==

@dlt.table(
    name="silver_event_hourly_stats",
    comment="Hourly event statistics for monitoring",
    table_properties={
        "quality": "silver",
        "layer": "silver"
    }
)
def silver_event_hourly_stats():
    """Aggregate events by hour for dashboards."""
    return (
        dlt.read("silver_events")
        .groupBy(
            date_trunc("hour",
col("event_timestamp")).alias("event_hour"),
            col("event_type")
        )
        .agg(
            count("*").alias("event_count"),
            countDistinct("user_id").alias("unique_users")
        )
    )
```

When to Use Raw Structured Streaming (Exception Cases)

Raw Structured Streaming should only be used when Lakeflow SDP cannot meet requirements:

Exception Case	Reason	Approval Required
Complex stateful operations	Custom state stores not supported in SDP	Platform Architect
Non-Delta sinks	Writing to external systems (JDBC, etc.)	Platform Architect
Custom watermark logic	Advanced windowing requirements	Platform Architect
Legacy migration	Existing Structured Streaming code	Data Steward

Raw Structured Streaming Best Practices (Exception Cases Only)

> **Note:** Compute policy enforcement is defined in CP-05. The patterns below show streaming-specific configuration.

```yaml
# Jobs compute configuration for raw Structured Streaming
resources:
  jobs:
    streaming_job:
      name: "[${bundle.target}] Legacy Streaming Job"

      # Continuous trigger for streaming
      trigger:
        periodic:
          interval: 1
          unit: HOURS

      # Unlimited retries for resilience
      max_retries: -1

      job_clusters:
        - job_cluster_key: streaming
          new_cluster:
            num_workers: 4  # Fixed - no auto-scaling!
            spark_version: "14.3.x-scala2.12"
            spark_conf:
              spark.streaming.stopGracefullyOnShutdown: "true"
            custom_tags:
              team: data-engineering
              workload_type: streaming

      tasks:
        - task_key: stream
          job_cluster_key: streaming
          notebook_task:
            notebook_path: ../src/streaming/legacy_pipeline.py
```

Rule ST-03: Disable auto-scaling for streaming workloads

```yaml
# ❌ WRONG: Auto-scaling causes state redistribution
new_cluster:
  autoscale:
    min_workers: 2
    max_workers: 8

# ✅ CORRECT: Fixed worker count
new_cluster:
  num_workers: 4
```

Rule ST-04: Ensure all streaming operations are idempotent

```python
def idempotent_foreach_batch(batch_df, batch_id):
    """Use MERGE for idempotent writes."""
    batch_df.createOrReplaceTempView("updates")
```

```
    spark.sql("""
        MERGE INTO silver.events AS target
        USING updates AS source
        ON target.event_id = source.event_id
        WHEN MATCHED THEN UPDATE SET *
        WHEN NOT MATCHED THEN INSERT *
    """)

# Streaming with idempotent writes
(spark.readStream
    .table("bronze.events")
    .writeStream
    .foreachBatch(idempotent_foreach_batch)
    .option("checkpointLocation", "/checkpoints/events")
    .trigger(processingTime="10 seconds")
    .start())
```

Raw Structured Streaming Checklist (Exception Cases)

- [ ] Exception approved by Platform Architect
- [ ] Using jobs compute (not all-purpose)
- [ ] Auto-scaling disabled (fixed worker count)
- [ ] Checkpoint location explicitly configured
- [ ] `foreachBatch` is idempotent (uses MERGE)
- [ ] No `display()` or `count()` in production code
- [ ] Graceful shutdown enabled
- [ ] Monitoring configured

Pipeline Monitoring

Lakeflow SDP Monitoring

Lakeflow SDP provides built-in monitoring through:

1. **Pipeline Dashboard** - Visual flow and metrics
2. **Event Log** - Detailed execution history
3. **Data Quality Metrics** - Expectation pass/fail rates
4. **System Tables** - Query `system.lakeflow.pipeline_*` tables

```
-- Query pipeline execution history
SELECT
    pipeline_name,
    update_id,
    state,
    start_time,
    end_time,
    TIMESTAMPDIFF(MINUTE, start_time, end_time) AS duration_minutes
FROM system.lakeflow.pipeline_events
WHERE pipeline_name = 'Event Streaming Pipeline'
```

```sql
ORDER BY start_time DESC
LIMIT 10;

-- Query data quality metrics
SELECT
    expectation_name,
    dataset_name,
    passed_records,
    failed_records,
    ROUND(100.0 * passed_records / (passed_records + failed_records), 2)
AS pass_rate
FROM system.lakeflow.flow_progress
WHERE pipeline_name = 'Event Streaming Pipeline'
ORDER BY timestamp DESC;
```

Validation Checklist

Lakeflow SDP Pipeline

- [ ] Using Lakeflow SDP (not raw Structured Streaming) unless exception approved
- [ ] Pipeline configured as serverless (preferred)
- [ ] Expectations defined for all streaming tables
- [ ] Quarantine table exists for invalid records
- [ ] Edition set to ADVANCED for expectations
- [ ] Change Data Feed enabled for downstream consumers
- [ ] Tags configured for cost allocation
- [ ] Notifications configured for failures
- [ ] root_path configured for organized assets

Raw Structured Streaming (Exception Cases Only)

- [ ] Exception approved and documented
- [ ] Using jobs compute (not all-purpose)
- [ ] Auto-scaling disabled
- [ ] Checkpoint location explicitly specified
- [ ] `foreachBatch` is idempotent
- [ ] No debug code (`display()`, `count()`)
- [ ] Monitoring configured

References

Lakeflow SDP (Primary)

- Lakeflow Spark Declarative Pipelines
- Lakeflow SDP Concepts
- Manage data quality with expectations
- Develop pipelines

- Configure pipelines
- Monitor pipelines

Structured Streaming (Exception Cases)

- Production Streaming Best Practices
- Structured Streaming Programming Guide